

Wave Function Collapse

Projet 801 — implémentation HPC

Nicolas Marano

12 mai 2026

- ① Le problème : reproduire un motif local
- ② L'algorithme WFC en six étapes
- ③ Architecture C++ et structures de données
- ④ Parallélisation : où, pourquoi, comment
- ⑤ Résultats : speedup et scalabilité
- ⑥ Démo
- ⑦ Conclusions

À partir d'un échantillon S (grille d'entiers), produire une grille G plus grande qui « ressemble localement » à S :

tout sous-bloc $N \times N$ de G doit apparaître dans S .

À partir d'un échantillon S (grille d'entiers), produire une grille G plus grande qui « ressemble localement » à S :

tout sous-bloc $N \times N$ de G doit apparaître dans S .

Applications : génération procédurale (jeux, level design), inpainting, synthèse de textures, motifs cellulaires.

À partir d'un échantillon S (grille d'entiers), produire une grille G plus grande qui « ressemble localement » à S :

tout sous-bloc $N \times N$ de G doit apparaître dans S .

Applications : génération procédurale (jeux, level design), inpainting, synthèse de textures, motifs cellulaires.

Difficulté : problème de satisfaction de contraintes globales ; naïvement exponentiel. WFC est une heuristique gloutonne.

L'algorithme en 6 étapes

- ➊ **Extraire** les tuiles $N \times N$ de S , dédupliquer, compter les fréquences
- ➋ **Pré-calculer** les compatibilités tuile $\times$ offset
- ➌ **Initialiser** la wave : chaque cellule = ensemble de toutes les tuiles
- ➍ **Sélectionner** la cellule d'entropie minimale
- ➎ **Choisir** une tuile (probabilité $\propto$ fréquence) et la poser
- ➏ **Propager** la contrainte aux voisins, puis (4)

L'algorithme en 6 étapes

- ➊ **Extraire** les tuiles $N \times N$ de S , dédupliquer, compter les fréquences
- ➋ **Pré-calculer** les compatibilités tuile $\times$ offset
- ➌ **Initialiser** la wave : chaque cellule = ensemble de toutes les tuiles
- ➍ **Sélectionner** la cellule d'entropie minimale
- ➎ **Choisir** une tuile (probabilité $\propto$ fréquence) et la poser
- ➏ **Propager** la contrainte aux voisins, puis (4)

Échec : si une cellule a un set vide $\rightarrow$ on retente avec un autre seed.

```
include/wfc/  
    Grid, Tile, Bitset, TileSet, OverlapRules, Wave  
    WFCSolver (interface)  
    solvers/  Serial / OMP / Kokkos  
src/  
apps/        wfc_serial, wfc_omp, wfc_kokkos, benchmark  
tests/       test_grid, test_tileset, test_overlap, test_solver  
samples/     binaires + multi-valeurs
```

```
include/wfc/  
  Grid, Tile, Bitset, TileSet, OverlapRules, Wave  
  WFCSolver (interface)  
  solvers/  Serial / OMP / Kokkos  
src/  
apps/      wfc_serial, wfc_omp, wfc_kokkos, benchmark  
tests/     test_grid, test_tileset, test_overlap, test_solver  
samples/   binaires + multi-valeurs
```

Cœur wfc_core agnostique du backend. Les libs OMP / Kokkos s'ajoutent par options CMake.

Bitset packé sur `uint64_t`

- AND/OR vectoriels 64 bits à la fois
- popcount via `__builtin_popcountll`
- first_set via `__builtin_ctzll`

Bitset packé sur `uint64_t`

- AND/OR vectoriels 64 bits à la fois
- popcount via `__builtin_popcountll`
- first_set via `__builtin_ctzll`

Wave = `vector<Bitset>` de taille `rows × cols`

- Accès toroïdal pour la cohérence avec l'extraction
- Atomic AND au niveau du mot pour la concurrence

Bitset packé sur `uint64_t`

- AND/OR vectoriels 64 bits à la fois
- popcount via `__builtin_popcountll`
- first_set via `__builtin_ctzll`

Wave = `vector<Bitset>` de taille `rows × cols`

- Accès toroïdal pour la cohérence avec l'extraction
- Atomic AND au niveau du mot pour la concurrence

OverlapRules : tableau plat indexé par $\text{tile_id} \times (2N - 1)^2 + \text{offset} \rightarrow$ bitset de tuiles compatibles

Où paralléliser ? Profil avant tout

Étape	Temps relatif
Extraction des tuiles	< 0.01 %
Règles d'adjacence	< 0.01 %
Sélection min-entropie	~85 %
Propagation	~12 %
RNG / divers	~3 %

Où paralléliser ? Profil avant tout

Étape	Temps relatif
Extraction des tuiles	< 0.01 %
Règles d'adjacence	< 0.01 %
Sélection min-entropie	~85 %
Propagation	~12 %
RNG / divers	~3 %

Surprise : la sélection domine, pas la propagation.

Leçon : mesurer avant d'optimiser.

OpenMP avec #pragma omp task

Sélection min-entropie

```
#pragma omp parallel
#pragma omp single
for (int k = 0; k < n_chunks; ++k) {
    #pragma omp task firstprivate(k, start, end)
    partials[k] = local_min(start, end);
}
// réduction en ordre de chunk → déterministe
```

OpenMP avec #pragma omp task

Sélection min-entropie

```
#pragma omp parallel
#pragma omp single
for (int k = 0; k < n_chunks; ++k) {
    #pragma omp task firstprivate(k, start, end)
    partials[k] = local_min(start, end);
}
// réduction en ordre de chunk → déterministe
```

Propagation BFS — une seule région parallel ouverte

```
#pragma omp parallel
while (!finished) {
    #pragma omp single
    for (chunks) #pragma omp task process(...);
    // implicit taskwait
```

Déterminisme à travers les backends

Conserver « même seed $\rightarrow$ même output » sur série / OMP / Kokkos :

- ❶ **Jitter d'entropie déterministe** : hash sur (cell_id, seed), pas de RNG consommé
- ❷ **Réduction ordonnée** : min en ordre de chunk, pas `reduction(min:)`
- ❸ **Atomics** : AND associatif $\rightarrow$ résultat invariant à l'ordre

Déterminisme à travers les backends

Conserver « même seed $\rightarrow$ même output » sur série / OMP / Kokkos :

- ❶ **Jitter d'entropie déterministe** : hash sur (cell_id, seed), pas de RNG consommé
- ❷ **Réduction ordonnée** : min en ordre de chunk, pas `reduction(min:)`
- ❸ **Atomics** : AND associatif $\rightarrow$ résultat invariant à l'ordre

Vérifié par diff byte-à-byte et par `test_solver` (run x2 avec même seed).

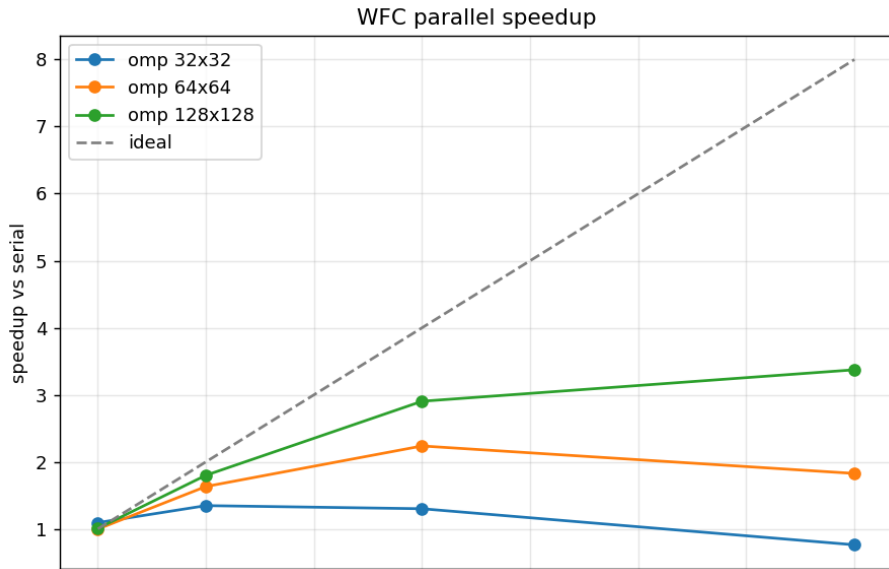
Speedup mesuré (128×128, sample binaire L=11)

Threads	Solve médian (s)	Speedup	Efficacité
Série	3.30	1.00 ×	100 %
OMP 1	3.28	1.00 ×	100 %
OMP 2	1.83	1.79 ×	90 %
OMP 4	1.14	2.88 ×	72 %
OMP 8	0.98	3.34 ×	42 %

Speedup mesuré (128×128, sample binaire L=11)

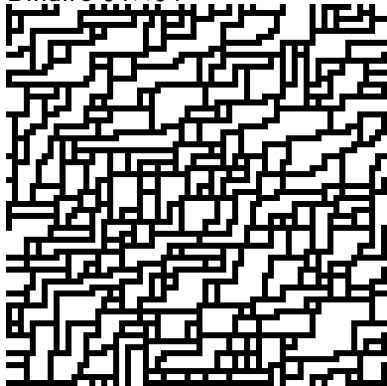
Threads	Solve médian (s)	Speedup	Efficacité
Série	3.30	1.00 ×	100 %
OMP 1	3.28	1.00 ×	100 %
OMP 2	1.83	1.79 ×	90 %
OMP 4	1.14	2.88 ×	72 %
OMP 8	0.98	3.34 ×	42 %

Plate-forme : g++ 13.3, AMD/Intel × 20 threads, WSL2 Ubuntu 24.04.

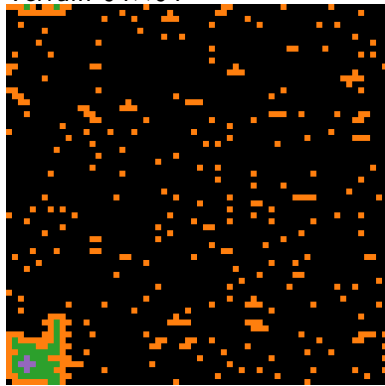


Démo : sorties générées

Binaire 64×64



Terrain 64×64



Labyrinthe 48×48



Multi-valeurs : extension gratuite

L'extension multi-valeurs n'a demandé **aucune modification du solveur** :

- valeurs en `uint8_t` (256 niveaux possibles) ;
- bitsets indexent des identifiants de tuiles, pas des valeurs ;
- le cas binaire est juste un cas particulier.

Multi-valeurs : extension gratuite

L'extension multi-valeurs n'a demandé **aucune modification du solveur** :

- valeurs en `uint8_t` (256 niveaux possibles) ;
- bitsets indexent des identifiants de tuiles, pas des valeurs ;
- le cas binaire est juste un cas particulier.

Adaptations limitées au rendu : palette qualitative 16 couleurs, scale configurable.

- Convention d'offset du README inverse de l'intuition → détecté par test de symétrie
- Première version OMP *plus lente* que serial (fork/join par niveau BFS)
- Min-entropie sous-estimée comme coût → profilage avant optimisation
- Déterminisme : RNG remplacé par hash sur (cell, seed)
- Échecs de convergence sur N grand : retry avec seed dérivé

- Trois backends opérationnels (serial / OMP / Kokkos), **déterministes**
- Speedup **3.3× à 8 threads** sur 128×128
- Multi-valeurs sans coût additionnel grâce au design en `uint8_t`
- 1.4k lignes de C++ + tests + benchmarks reproductibles

- Trois backends opérationnels (serial / OMP / Kokkos), **déterministes**
- Speedup **3.3× à 8 threads** sur 128×128
- Multi-valeurs sans coût additionnel grâce au design en `uint8_t`
- 1.4k lignes de C++ + tests + benchmarks reproductibles

Pistes : backtracking, heuristique avec PQ, symétries de tuiles, backend GPU via Kokkos.

Questions ?

Code, samples, benchmarks et rapport disponibles dans le repo.